

Phase 2 First-Pass Post-Mortem

Rendering Carve-Out from main.js

Date: 2026-09-14

Commit: ab04ab7 on main

Scope: Prove the extraction pattern with three small, well-scoped render surfaces. This is the smallest useful chunk of Phase 2 — not the whole carve-out.

The one-paragraph version

`main.js` is 22,600 lines because roughly half of it is HTML template strings inlined into render functions that all live in the same file. Phase 2 is a long march to pull those out. This first pass extracts three small render surfaces to prove the pattern works. Each extracted module is a pure `(state) → HTML string` function with no closures over `main.js` internals. Every existing callsite still works via a thin alias. Zero user-facing change. Byte-for-byte identical HTML output. Future extractions follow the same recipe.

Why we're doing this at all

`main.js` is one giant closure. Every render function inside it reads local variables — `activeRole`, `_selectedEventId`, DOM refs, and dozens of other pieces of state — that only exist inside that outer closure. Adding a new feature that touches rendering means editing a 22,600-line file. Reviewing that diff means reading a 22,600-line file. Future engineers onboarding to the codebase have to swim through it every time.

The fix is not React. Not a framework migration. Not a rewrite. Just: **pull render functions into their own modules, one at a time, keep the API surface identical, and let `main.js` shrink over months of small extractions.**

The extraction recipe

For each render function extracted, the steps are:

1. Read the function body. Identify every variable it closes over (state, other functions, imported modules).
2. Turn every closed-over reference into either an explicit parameter or an explicit import in the new module.
3. Create `src/render/<name>.js` with the extracted function as a named export.
4. In `main.js`, `import` the new function and add a `const _oldName = newName` alias at the same place the original function used to live.
5. Delete the old function body.
6. Run the build. Both safety nets (Phase 1 event-mutation policy + Phase 3 tenant-stamping) plus Vite must all pass.

The thin alias is important — it means the ~10 existing callsites of a function like `_renderMarkerFilterChips(event)` continue to work without touching the callsite. The extraction stays reversible until every callsite is comfortable enough with the new module for the alias to be removed.

The three surfaces extracted

1. Marker filter chips

File: `src/render/marker_filter_chips.js`

Callsites: 2 in `main.js` (`_renderMarkerFilterChips` at line 9875, `_renderMarkerFilterChipsBody` at line 15691)

Small filter chip strip rendered in the debrief modal + Palantir closed panel. Two variants (full section + body-only). Also exports `defaultMarkerFilters()` — the factory for the default per-event filter state.

The lazy `_markerFilters` stamp uses `mutateEvent` from `events.js`, so the Phase 1 mutation policy is preserved when extracted.

2. Escalation progress badge

File: `src/render/escalation_progress_badge.js`
Callsites: 3 in `main.js` (operator log rows, receiver inbox card, history views)

Compact pill showing the post-acknowledgment progress state of an escalation. `in-progress` (cyan), `resolved` (green), `blocked` (amber + reason). Empty when no progress state is set.

Cleanest of the three extractions — pure function, no state closures, no imports needed except the module's own.

3. Reports filter chips

File: `src/render/reports_filter_chips.js`
Callsites: 1 in `main.js` (`_renderReportsFilterChips` at line 21419)

Filter strip on top of the receiver Reports tab. `<select>` chips for site / kommune / politikreds / region / classification / domain / time range. Each chip only appears when the pool has 2+ distinct values on that dimension.

Also exports `REPORTS_FILTER_DEFAULTS` — the default filter state that `main.js` uses at boot and on every role change to reset. Moving the defaults with the render function keeps the "what does clean state look like" invariant in one place.

Also fixed while I was in there: 5 Phase-1 misses

The Phase 1 audit grep matched `event.foo = ...` writes. It missed 5 places where `main.js` used the shorter variable name `ev` inside `EVENTS.forEach(ev => ...)` and `for (const ev of EVENTS)` loops.

Line	What it was doing
<code>main.js:98</code>	Seeding PIR reports for closed events at boot
<code>main.js:7821</code>	Initialising <code>contributingSensors</code> array for the WebSocket NN detection path
<code>main.js:17729</code>	Marker filter chip click flipping filter state
<code>main.js:21669</code>	Recording dispatch outcome on operator confirmation
<code>main.js:22201</code>	Promoting an observer to actor on the case file

All five now route through `mutateEvent`, `appendEventArray`, `attachPostIncidentReport`, or `setEventMapKey` as appropriate.

And extended the safety-net regex in `scripts/check-event-mutations.mjs` to catch both `event.` and `ev.` prefixes. This class of regression can't recur — build fails if anyone reintroduces it.

What the numbers look like

Metric	Before Phase 2	After first pass
Modules in the build	81	84
Render functions extracted	0	3
Render surfaces still inline in <code>main.js</code>	~40	~37
<code>main.js</code> line count	22,610	22,510

Direct event writes caught by safety net	event. prefix only	event. + ev.
Phase-1 misses patched	—	5
Build clean	Yes	Yes
User-facing behaviour	Baseline	Identical

Line count barely moved (100 lines) — because the extracted functions are small. Bigger targets like `renderDetailPanel` (350+ lines), `_renderStep3ActiveEngagement` and siblings (400 lines each), `renderConfig` (200 lines), `renderReceiverView` (~500 lines) will shift the number a lot more when they land.

Why this order

The three extracted surfaces were chosen for one reason: **lowest possible risk while proving the pattern.**

- **Marker filter chips:** tiny (25 LOC each), one closed-over reference (`_defaultMarkerFilters` and `mutateEvent`), clean input/output boundary.
- **Escalation progress badge:** tinier (13 LOC), zero closed-over references, purest function of the three.
- **Reports filter chips:** bigger (~65 LOC) but only closes over `SITES` (globally imported) and `_REPORTS_FILTER_DEFAULTS` (which came with it to the new module).

If the pattern was going to break, it would have broken on one of these three. It didn't. The build still passes both safety nets. HTML output is byte-for-byte identical (verified by the alias + the extracted function producing the same string).

Every future extraction now has a proven template to follow. Bigger surfaces can land the same way.

What comes next

Not in this session. When Phase 2 resumes:

Tier 2 (moderate, single-owner state):

- `_renderStep3ActiveEngagement` , `_renderStep40OutcomeConfirm` , `_renderStep5PostIncidentHandoff` , `_renderStep6CloseEvent` — the receiver Case File step renderers. All take (event, activeRole) , all live in the ~19000 line range. Natural cluster for a `src/render/case_file_steps.js` module.
- `_renderPostIncidentReportPanel` , `_renderPirEventChain` , `_renderPirContributorChapters` — the PIR panel cluster.

Tier 3 (larger, more state entanglement):

- `renderDetailPanel` , `renderPalantirClosedPanel` , `renderEventWorkspace` — the biggest render surfaces. Each closes over selection state, role state, DOM refs. Extraction requires more careful parameter threading.

Tier 4 (state-heavy):

- `renderReceiverView` , `renderReceiverParentLanding` — the top-level receiver render orchestrators. May not extract cleanly until the state they touch also moves out of main.js.

None of this is required for anything else to work. Phase 2 is pure iteration-speed investment. It runs in parallel with Azure work when Azure lands.

The verdict

Pattern proven. Three modules landed. Build clean. Zero regressions detectable.

Phase 2 is a marathon, not a sprint — 40+ render surfaces to go. But every future extraction is now mechanical: read the function, identify what it closes over, extract, alias, ship.

Phases 1 + 2 (first pass) + 3 all landed within a two-day window. The three architectural risks from the deep-dive read are all now either fully addressed (Phase 1, Phase 3) or actively being unwound (Phase 2). The platform is materially more prepared for the Azure substrate than it was Monday morning.